

Document number	P3356R0
Date	2024-07-13
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Library Evolution Working Group (LEWG)

non_invalidating_vector

Table of contents

- non_invalidating_vector
 - Abstract
 - Motivational Example
 - Prior Work
 - Potential Improvements
 - Alternative
 - Wording
 - Alternate Wording
 - Impact on the standard
 - Implementation
 - References

Abstract

Add a non invalidating vector and a non invalidating reference of `std::vector` to the standard template library in order to make it easier for programmers to proactively manage their invalidation safety concerns.

Motivational Example

```
void non_invalidating(non_invalidating_vector_ref<int> niv, const std::vector<int>
{
    // no worry of niv invalidating
    // no worry of cv invalidating
}

int main()
{
    std::vector<int> vs;
    non_invalidating_vector<int> niv;// free of invalidation concerns
    non_invalidating(vs, vs);
    non_invalidating(niv, vs);
    [](non_invalidating_vector_ref<int> vs)
    {
        // no worry of vs invalidating
    }(vs);
    for (auto &s : vs) {
        [](non_invalidating_vector_ref<int> vs)
        {
            // no worry of vs invalidating
        }(vs);
    }
    return 0;
}
```

non_invalidating_vector_ref

`non_invalidating_vector_ref` is just a pure reference to a `std::vector` that allow the programmer to call all of the non invalidating [member] functions and deletes all of the invalidating [member] functions. It also can be narrowed to `const std::vector&` which is even more restricted. It's primarily meant to be used as parameters but like lvalue references in general can be used in a similar manner.

```

template<
    class T,
    class Allocator = std::allocator<T>
>
class non_invalidating_vector_ref
{
private:
    actual_type& ref;
public:
    // constructors reflect this being just a pure reference type
    constexpr non_invalidating_vector_ref() = delete;
    constexpr non_invalidating_vector_ref(const non_invalidating_vector_ref&) = delete;
    constexpr non_invalidating_vector_ref(non_invalidating_vector_ref&&) = default;
    constexpr non_invalidating_vector_ref operator=(const non_invalidating_vector_ref&) = delete;
    constexpr non_invalidating_vector_ref operator=(const non_invalidating_vector_ref&&) = delete;

    constexpr non_invalidating_vector_ref(actual_type& reference) : ref{reference} {}
    // all non invalidating methods are proxied
    constexpr reference operator[]( size_type pos ) const
    {
        return ref[pos];
    }
    constexpr const_reference operator[]( size_type pos ) const
    {
        return ref[pos];
    }
    constexpr size_type size() const
    {
        return ref.size();
    }
    // all invalidating methods are deleted
    constexpr void resize( size_type count ) = delete;
    // can narrow scope to just the const members
    operator const actual_type&() const
    {
        return ref;
    }
};

```

non_invalidating_vector

`non_invalidating_vector` is a container adaptor like `stack`, `queue` and others. After being constructed, it can no longer be invalidated. It allows the programmer to call all of the non invalidating [member] functions and deletes all of the invalidating [member] functions. It also

can be narrowed to `non_invalidating_vector_ref` and also to `const std::vector&` which is even more restricted. It's primarily meant to be used with the C++23 range based constructor which allows it to be initialized in an ever growing number of ways.

```

template<
    class T,
    class Allocator = std::allocator<T>,
    class Container = std::vector<T, Allocator>
>
class non_invalidating_vector
{
private:
    actual_type inner;
public:
    // has all of the std::vector constructors
    constexpr non_invalidating_vector() noexcept(noexcept(Allocator()))
        : inner() {}
    constexpr explicit non_invalidating_vector( const Allocator& alloc ) noexcept
        : inner(alloc) {}
    constexpr non_invalidating_vector( size_type count, const T& value, const Alloc
        : inner(count, value, alloc) {}
    explicit non_invalidating_vector( size_type count, const Allocator& alloc = Al
        : inner(count, alloc) {}
    template< class InputIt >
    constexpr non_invalidating_vector( InputIt first, InputIt last, const Allocat
        : inner(first, last, alloc) {}
    constexpr non_invalidating_vector( const non_invalidating_vector& other )
        : inner(other) {}
    constexpr non_invalidating_vector( const non_invalidating_vector& other, const
        : inner(other, alloc) {}
    constexpr non_invalidating_vector( non_invalidating_vector&& other ) noexcept
        : inner(other) {}
    constexpr non_invalidating_vector( non_invalidating_vector&& other, const Alloc
        : inner(other, alloc) {}
    constexpr non_invalidating_vector( std::initializer_list<T> init, const Allocat
        : inner(init, alloc) {}
    template< class R >
    constexpr non_invalidating_vector( std::from_range_t, R&& rg, const Allocator&
        : inner(std::from_range, rg, alloc) {}
    // all non invalidating methods are proxied
    constexpr reference operator[]( size_type pos )
    {
        return inner[pos];
    }
    constexpr const_reference operator[]( size_type pos ) const
    {
        return inner[pos];
    }
    constexpr size_type size() const
    {
        return inner.size();
    }
    // all invalidating methods are deleted

```

```
constexpr void resize( size_type count ) = delete;
// compatible with non_invalidating_vector_ref
operator non_invalidating_vector_ref<T, Allocator>()
{
    return non_invalidating_vector_ref<T, Allocator>{inner};
}
// can narrow scope to just the const members
operator const actual_type&() const
{
    return inner;
}
};
```

Prior Work

This proposal was created to fill the need stated in the [A framework for Profiles development](#) ^[1] proposal. While [Guidelines Support Library](#) ^[2] ^[3] still does not have a `dyn_array` or a `static_array`, `non_invalidating_vector_ref` and `non_invalidating_vector` together are meant to provide equivalent intended functionality as `dyn_array` but instead of using a new collection, it builds upon what we already have, `std::vector`. This proposal, while only the beginning, should get us thinking that it would be valuable from a defensive programming perspective to have non invalidating views of most of our containers and container adaptors eventually.

A framework for Profiles development ^[1:1]

Why 99%? Well, I could have said 90% and still made the point. What might be an example? Consider ... containers without invalidating operations (e.g., `gsl::dyn_array` rather than `vector`)

"3.3. Profile: Concurrency"

- **Observation:** ...
 - **Invalidation:** use ... containers without invalidation (e.g., `gsl::dyn_array`) to pass information between threads.

Potential Improvements

Given any container it would be good to know what is its `non_invalidating` type and given a container instance, a consistently named member function to get an instance of or reference to the `non_invalidating` type.

```
namespace std {
    template<class T, class Allocator = allocator<T>>
    class vector {
    public:
        using non_invalidating_type = non_invalidating_vector<T, Allocator>;
        non_invalidating_type non_invalidating_view() const noexcept
        {
            return non_invalidating_type(*this);
        }
    }
}
```

This would allow the user of the container to autocomplete their way to the invalidation free safety.

Alternative

Refactor vector

Instead of adding two classes to any container, effectively tripling portions of the specification, inheritance could be used instead to split any given container into its invalidating and non invalidating variants

current <code>std::vector</code>	
non_invalidating_vector base class	future <code>std::vector</code>
• <code>get_allocator</code> • <code>at</code> • <code>operator[]</code> • <code>front</code> • <code>back</code> • <code>data</code> • <code>begin</code> • <code>cbegin</code> • <code>end</code> • <code>cend</code> • <code>rbegin</code> • <code>crbegin</code> • <code>rend</code> • <code>crend</code> • <code>empty</code> • <code>size</code> • <code>max_size</code> • <code>capacity</code>	• <code>operator=</code> • <code>assign</code> • <code>assign_range</code> • <code>shrink_to_fit</code> • <code>clear</code> • <code>insert</code> • <code>insert_range</code> • <code>emplace</code> • <code>erase</code> • <code>push_back</code> • <code>emplace_back</code> • <code>append_range</code> • <code>pop_back</code> • <code>resize</code> • <code>swap</code>

This would elevate the usage of this base class as being just a cast thus making it more consistent with a `const` cast.

	non_invalidating_vector non_invalidating_vector_ref	inheritance non_invalidating_vector base class
PRO(s)	• pure addition	• split one class in two
CON(s)	• requires 2 full classes	• modifies collection class

This would require refactoring any given collection by injecting a base class and relocating the non invalidating member functions to said base class. Since the resolved collection of methods would remain the same in the final collection, no breakage is expected.

Wording

24.3.11 Class template non_invalidating_vector_ref **[non_invalidating_vector_ref]**

24.3.11.1 Overview [non_invalidating_vector_ref.overview]

¹ A non_invalidating_vector_ref is a sequence container that supports (amortized) constant time insert and erase operations at the end; insert and erase in the middle take linear time. Storage management is handled automatically, though hints can be given to improve efficiency.

² A non_invalidating_vector_ref meets all of the requirements of a container (24.2.2.2), of a reversible container (24.2.2.3), of an allocator-aware container (24.2.2.5), of a sequence container, including most of the optional sequence container requirements (24.2.4), and, for an element type other than bool, of a contiguous container (24.2.2.2). The exceptions are the push_front, prepend_range, pop_front, and emplace_front member functions, which are not provided. Descriptions are provided here only for operations on non_invalidating_vector_ref that are not described in one of these tables or for operations where there is additional semantic information.

³ The types iterator and const_iterator meet the constexpr iterator requirements (25.3.1).

```

namespace std {
    template<class T, class Allocator = allocator<T>>
    class non_invalidating_vector_ref {
    public:
        // types
        using actual_type = std::vector<T, Allocator>;
        using value_type = T;
        using allocator_type = Allocator;
        using pointer = typename allocator_traits<Allocator>::pointer;
        using const_pointer = typename allocator_traits<Allocator>::const_pointer;
        using reference = value_type&;
        using const_reference = const value_type&;
        using size_type = implementation-defined ; // see 24.2
        using difference_type = implementation-defined ; // see 24.2
        using iterator = implementation-defined ; // see 24.2
        using const_iterator = implementation-defined ; // see 24.2
        using reverse_iterator = std::reverse_iterator<iterator>;
        using const_reverse_iterator = std::reverse_iterator<const_iterator>;

        // 24.3.11.2, construct/copy/destroy
        constexpr non_invalidating_vector_ref() = delete;
        constexpr non_invalidating_vector_ref(const non_invalidating_vector_ref&)
        constexpr non_invalidating_vector_ref(non_invalidating_vector_ref&&) = delete;
        constexpr non_invalidating_vector_ref operator=(const non_invalidating_vector_ref&) = delete;
        constexpr non_invalidating_vector_ref operator=(const non_invalidating_vector_ref&&) = delete;
        constexpr non_invalidating_vector_ref(actual_type& reference);
        constexpr ~non_invalidating_vector_ref();
        template<class InputIterator>
        constexpr void assign(InputIterator first, InputIterator last) = delete;
        template<container-compatible-range <T> R>
        constexpr void assign_range(R&& rg) = delete;
        constexpr void assign(size_type n, const T& u) = delete;
        constexpr void assign(initializer_list<T>) = delete;
        constexpr allocator_type get_allocator() const noexcept;

        // iterators
        constexpr iterator begin() noexcept;
        constexpr const_iterator begin() const noexcept;
        constexpr iterator end() noexcept;
        constexpr const_iterator end() const noexcept;
        constexpr reverse_iterator rbegin() noexcept;
        constexpr const_reverse_iterator rbegin() const noexcept;
        constexpr reverse_iterator rend() noexcept;
        constexpr const_reverse_iterator rend() const noexcept;

        constexpr const_iterator cbegin() const noexcept;
        constexpr const_iterator cend() const noexcept;
        constexpr const_reverse_iterator crbegin() const noexcept;
        constexpr const_reverse_iterator crend() const noexcept;

```

```

// 24.3.11.3, capacity
[[nodiscard]] constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;
constexpr size_type capacity() const noexcept;
constexpr void resize(size_type sz) = delete;
constexpr void resize(size_type sz, const T& c) = delete;
constexpr void reserve(size_type n) = delete;
constexpr void shrink_to_fit() = delete;

// element access
constexpr reference operator[](size_type n);
constexpr const_reference operator[](size_type n) const;
constexpr const_reference at(size_type n) const;
constexpr reference at(size_type n);
constexpr reference front();
constexpr const_reference front() const;
constexpr reference back();
constexpr const_reference back() const;

// 24.3.11.4, data access
constexpr T* data() noexcept;
constexpr const T* data() const noexcept;

// 24.3.11.5, modifiers
template<class... Args> constexpr reference emplace_back(Args&&... args) =
constexpr void push_back(const T& x) = delete;
constexpr void push_back(T&& x) = delete;
template<container-compatible-range <T> R>
constexpr void append_range(R&& rg) = delete;
constexpr void pop_back() = delete;

template<class... Args> constexpr iterator emplace(const_iterator position,
constexpr iterator insert(const_iterator position, const T& x) = delete;
constexpr iterator insert(const_iterator position, T&& x) = delete;
constexpr iterator insert(const_iterator position, size_type n, const T& x) = delete;
template<class InputIterator>
constexpr iterator insert(const_iterator position,
    InputIterator first, InputIterator last) = delete;
template<container-compatible-range <T> R>
constexpr iterator insert_range(const_iterator position, R&& rg) = delete;
constexpr iterator insert(const_iterator position, initializer_list<T> il) = delete;
constexpr iterator erase(const_iterator position) = delete;
constexpr iterator erase(const_iterator first, const_iterator last) = delete;
constexpr void swap(non_invalidating_vector_ref&
    noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value
    allocator_traits<Allocator>::is_always_equal::value) = delete;

```

```

constexpr void clear() noexcept = delete;

operator const actual_type&() const;
};

template<class InputIterator, class Allocator = allocator<iter-value-type <InputIt
non_invalidating_vector_ref(InputIterator, InputIterator, Allocator = Allocator())
-> non_invalidating_vector_ref<iter-value-type <InputIterator>, Allocator>;

template<ranges::input_range R, class Allocator = allocator<ranges::range_value_t<
non_invalidating_vector_ref(from_range_t, R&&, Allocator = Allocator())
-> non_invalidating_vector_ref<ranges::range_value_t<R>, Allocator>;
}

```

⁴ An incomplete type T may be used when instantiating non_invalidating_vector_ref if the allocator meets the allocator completeness requirements (16.4.4.6.2). T shall be complete before any member of the resulting specialization of non_invalidating_vector_ref is referenced.

24.3.11.2 Constructors [non_invalidating_vector_ref.cons]

```
constexpr explicit non_invalidating_vector_ref(const Allocator&) noexcept;
```

¹ Effects: Constructs an empty non_invalidating_vector_ref, using the specified allocator.

² Complexity: Constant.

```
constexpr explicit non_invalidating_vector_ref(size_type n, const Allocator& = All
```

³ Preconditions: T is Cpp17DefaultInsertable into *this.

⁴ Effects: Constructs a non_invalidating_vector_ref with n default-inserted elements using the specified allocator.

⁵ Complexity: Linear in n.

```
constexpr non_invalidating_vector_ref(size_type n, const T& value,
constexpr Allocator& = Allocator());
```

⁶ Preconditions: T is Cpp17CopyInsertable into *this.

⁷ Effects: Constructs a `non_invalidating_vector_ref` with `n` copies of `value`, using the specified allocator.

⁸ Complexity: Linear in `n`.

```
template<class InputIterator>
constexpr non_invalidating_vector_ref(InputIterator first, InputIterator last,
    const Allocator& = Allocator());
```

⁹ Effects: Constructs a `non_invalidating_vector_ref` equal to the range `[first, last)`, using the specified allocator.

¹⁰ Complexity: Makes only `N` calls to the copy constructor of `T` (where `N` is the distance between `first` and `last`) and no reallocations if iterators `first` and `last` are of forward, bidirectional, or random access categories. It makes order `N` calls to the copy constructor of `T` and order $\log N$ reallocations if they are just input iterators.

```
template<container-compatible-range <T> R>
constexpr non_invalidating_vector_ref(from_range_t, R&& rg, const Allocator& = All
```

¹¹ Effects: Constructs a `non_invalidating_vector_ref` object with the elements of the range `rg`, using the specified allocator.

¹² Complexity: Initializes exactly `N` elements from the results of dereferencing successive iterators of `rg`, where `N` is `ranges::distance(rg)`. Performs no reallocations if `R` models `ranges::forward_range` or `ranges::sized_range`; otherwise, performs order $\log N$ reallocations and order `N` calls to the copy or move constructor of `T`.

24.3.11.3 Capacity [`non_invalidating_vector_ref.capacity`]

```
constexpr size_type capacity() const noexcept;
```

¹ Returns: The total number of elements that the `non_invalidating_vector_ref` can hold without requiring reallocation.

² Complexity: Constant time.

24.3.11.4 Data [`non_invalidating_vector_ref.data`]

```
constexpr T* data() noexcept;  
constexpr const T* data() const noexcept;
```

¹ Returns: A pointer such that `[data(), data() + size())` is a valid range. For a non-empty `non_invalidating_vector` `ref`, `data() == addressof(front())` is true.

² Complexity: Constant time.

24.3.11 Class template `non_invalidating_vector` [`non_invalidating_vector`]

24.3.11.1 Overview [`non_invalidating_vector.overview`]

¹ A `non_invalidating_vector` is a sequence container that supports (amortized) constant time insert and erase operations at the end; insert and erase in the middle take linear time. Storage management is handled automatically, though hints can be given to improve efficiency.

² A `non_invalidating_vector` meets all of the requirements of a container (24.2.2.2), of a reversible container (24.2.2.3), of an allocator-aware container (24.2.2.5), of a sequence container, including most of the optional sequence container requirements (24.2.4), and, for an element type other than `bool`, of a contiguous container (24.2.2.2). The exceptions are the `push_front`, `prepend_range`, `pop_front`, and `emplace_front` member functions, which are not provided. Descriptions are provided here only for operations on `non_invalidating_vector` that are not described in one of these tables or for operations where there is additional semantic information.

³ The types `iterator` and `const_iterator` meet the `constexpr` iterator requirements (25.3.1).

```

namespace std {
    template<class T, class Allocator = allocator<T>>
    class non_invalidating_vector {
    public:
        // types
        using actual_type = std::vector<T, Allocator>;
        using value_type = T;
        using allocator_type = Allocator;
        using pointer = typename allocator_traits<Allocator>::pointer;
        using const_pointer = typename allocator_traits<Allocator>::const_pointer;
        using reference = value_type&;
        using const_reference = const value_type&;
        using size_type = implementation-defined ; // see 24.2
        using difference_type = implementation-defined ; // see 24.2
        using iterator = implementation-defined ; // see 24.2
        using const_iterator = implementation-defined ; // see 24.2
        using reverse_iterator = std::reverse_iterator<iterator>;
        using const_reverse_iterator = std::reverse_iterator<const_iterator>;

        // 24.3.11.2, construct/copy/destroy
        constexpr non_invalidating_vector() noexcept(noexcept(Allocator())) : non_
        constexpr explicit non_invalidating_vector(const Allocator&) noexcept;
        constexpr explicit non_invalidating_vector(size_type n, const Allocator& =
        constexpr non_invalidating_vector(size_type n, const T& value, const Alloc
        template<class InputIterator>
        constexpr non_invalidating_vector(InputIterator first, InputIterator last,
        template<container-compatible-range <T> R>
        constexpr non_invalidating_vector(from_range_t, R&& rg, const Allocator& =
        constexpr non_invalidating_vector(const non_invalidating_vector& x);
        constexpr non_invalidating_vector(non_invalidating_vector&&) noexcept;
        constexpr non_invalidating_vector(const non_invalidating_vector&, const ty
        constexpr non_invalidating_vector(non_invalidating_vector&&, const type_ic
        constexpr non_invalidating_vector(initializer_list<T>, const Allocator& =
        constexpr ~non_invalidating_vector();
        constexpr non_invalidating_vector& operator=(const non_invalidating_vector
        constexpr non_invalidating_vector& operator=(non_invalidating_vector&& x)
            noexcept(allocator_traits<Allocator>::propagate_on_container_move_assi
            allocator_traits<Allocator>::is_always_equal::value) = delete;
        constexpr non_invalidating_vector& operator=(initializer_list<T>) = delete;
        template<class InputIterator>
        constexpr void assign(InputIterator first, InputIterator last) = delete;
        template<container-compatible-range <T> R>
        constexpr void assign_range(R&& rg) = delete;
        constexpr void assign(size_type n, const T& u) = delete;
        constexpr void assign(initializer_list<T>) = delete;
        constexpr allocator_type get_allocator() const noexcept;

        // iterators
        constexpr iterator begin() noexcept;

```

```

constexpr const_iterator begin() const noexcept;
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;
constexpr reverse_iterator rbegin() noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator rend() noexcept;
constexpr const_reverse_iterator rend() const noexcept;

constexpr const_iterator cbegin() const noexcept;
constexpr const_iterator cend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// 24.3.11.3, capacity
[[nodiscard]] constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;
constexpr size_type capacity() const noexcept;
constexpr void resize(size_type sz) = delete;
constexpr void resize(size_type sz, const T& c) = delete;
constexpr void reserve(size_type n) = delete;
constexpr void shrink_to_fit() = delete;

// element access
constexpr reference operator[](size_type n);
constexpr const_reference operator[](size_type n) const;
constexpr const_reference at(size_type n) const;
constexpr reference at(size_type n);
constexpr reference front();
constexpr const_reference front() const;
constexpr reference back();
constexpr const_reference back() const;

// 24.3.11.4, data access
constexpr T* data() noexcept;
constexpr const T* data() const noexcept;

// 24.3.11.5, modifiers
template<class... Args> constexpr reference emplace_back(Args&&... args) =
constexpr void push_back(const T& x) = delete;
constexpr void push_back(T&& x) = delete;
template<container-compatible-range <T> R>
constexpr void append_range(R&& rg) = delete;
constexpr void pop_back() = delete;

template<class... Args> constexpr iterator emplace(const_iterator position
constexpr iterator insert(const_iterator position, const T& x) = delete;
constexpr iterator insert(const_iterator position, T&& x) = delete;
constexpr iterator insert(const_iterator position, size_type n, const T& x) = delete;

```



```

template<class InputIterator>
constexpr iterator insert(const_iterator position,
    InputIterator first, InputIterator last) = delete;
template<container-compatible-range <T> R>
constexpr iterator insert_range(const_iterator position, R&& rg) = delete;
constexpr iterator insert(const_iterator position, initializer_list<T> il)
constexpr iterator erase(const_iterator position) = delete;
constexpr iterator erase(const_iterator first, const_iterator last) = delete;
constexpr void swap(non_invalidating_vector&)
    noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value
    allocator_traits<Allocator>::is_always_equal::value) = delete;

constexpr void clear() noexcept = delete;

operator non_invalidating_vector_ref<T, Allocator>();
operator const actual_type&() const;
};

template<class InputIterator, class Allocator = allocator<iter-value-type <InputIterator>>
non_invalidating_vector(InputIterator, InputIterator, Allocator = Allocator())
    -> non_invalidating_vector<iter-value-type <InputIterator>, Allocator>;

template<ranges::input_range R, class Allocator = allocator<ranges::range_value_t<
non_invalidating_vector(from_range_t, R&&, Allocator = Allocator())
    -> non_invalidating_vector<ranges::range_value_t<R>, Allocator>;
}

```

⁴ An incomplete type T may be used when instantiating `non_invalidating_vector` if the allocator meets the allocator completeness requirements (16.4.4.6.2). T shall be complete before any member of the resulting specialization of `non_invalidating_vector` is referenced.

24.3.11.2 Constructors [non_invalidating_vector.cons]

```
constexpr explicit non_invalidating_vector(const Allocator&) noexcept;
```

¹ Effects: Constructs an empty `non_invalidating_vector`, using the specified allocator.

² Complexity: Constant.

```
constexpr explicit non_invalidating_vector(size_type n, const Allocator& = Allocator)
```

³ Preconditions: T is Cpp17DefaultInsertable into *this.

⁴ Effects: Constructs a non_invalidating_vector with n default-inserted elements using the specified allocator.

⁵ Complexity: Linear in n.

```
constexpr non_invalidating_vector(size_type n, const T& value,  
    const Allocator& = Allocator());
```

⁶ Preconditions: T is Cpp17CopyInsertable into *this.

⁷ Effects: Constructs a non_invalidating_vector with n copies of value, using the specified allocator.

⁸ Complexity: Linear in n.

```
template<class InputIterator>  
constexpr non_invalidating_vector(InputIterator first, InputIterator last,  
    const Allocator& = Allocator());
```

⁹ Effects: Constructs a non_invalidating_vector equal to the range [first, last), using the specified allocator.

¹⁰ Complexity: Makes only N calls to the copy constructor of T (where N is the distance between first and last) and no reallocations if iterators first and last are of forward, bidirectional, or random access categories. It makes order N calls to the copy constructor of T and order log N reallocations if they are just input iterators.

```
template<container-compatible-range <T> R>  
constexpr non_invalidating_vector(from_range_t, R&& rg, const Allocator& = Allocat
```

¹¹ Effects: Constructs a non_invalidating_vector object with the elements of the range rg, using the specified allocator.

¹² Complexity: Initializes exactly N elements from the results of dereferencing successive iterators of rg, where N is ranges::distance(rg). Performs no reallocations if R models ranges::forward_range or ranges::sized_range; otherwise, performs order log N reallocations and order N calls to the copy or move constructor of T.

24.3.11.3 Capacity [non_invalidating_vector.capacity]

```
constexpr size_type capacity() const noexcept;
```

¹ Returns: The total number of elements that the `non_invalidating_vector` can hold without requiring reallocation.

² Complexity: Constant time.

24.3.11.4 Data [non_invalidating_vector.data]

```
constexpr T* data() noexcept;  
constexpr const T* data() const noexcept;
```

¹ Returns: A pointer such that `[data(), data() + size())` is a valid range. For a non-empty `non_invalidating_vector`, `data() == addressof(front())` is true.

² Complexity: Constant time.

²⁰⁷⁾ `reserve()` uses `Allocator::allocate()` which can throw an appropriate exception.

Alternate Wording

24.3.11 Class template `non_invalidating_vector` [non_invalidating_vector]

24.3.11.1 Overview [non_invalidating_vector.overview]

¹ A `non_invalidating_vector` is a sequence container. Storage management is handled automatically, though hints can be given to improve efficiency.

² A `non_invalidating_vector` meets all of the requirements of a container (24.2.2.2), of a reversible container (24.2.2.3), of an allocator-aware container (24.2.2.5), of a sequence container, including most of the optional sequence container requirements (24.2.4), and, for an element type other than `bool`, of a contiguous container (24.2.2.2). The exceptions are the `push_front`, `prepend_range`, `pop_front`, and `emplace_front` member functions, which are not provided. Descriptions are provided here only for operations on `non_invalidating_vector` that are not described in one of these tables or for operations where there is additional semantic information.

³ The types `iterator` and `const_iterator` meet the `constexpr` iterator requirements (25.3.1).

```

namespace std {
    template<class T, class Allocator = allocator<T>>
    class non_invalidating_vector {
    public:
        // types
        using actual_type = std::vector<T, Allocator>;
        using value_type = T;
        using allocator_type = Allocator;
        using pointer = typename allocator_traits<Allocator>::pointer;
        using const_pointer = typename allocator_traits<Allocator>::const_pointer;
        using reference = value_type&;
        using const_reference = const value_type&;
        using size_type = implementation-defined ; // see 24.2
        using difference_type = implementation-defined ; // see 24.2
        using iterator = implementation-defined ; // see 24.2
        using const_iterator = implementation-defined ; // see 24.2
        using reverse_iterator = std::reverse_iterator<iterator>;
        using const_reverse_iterator = std::reverse_iterator<const_iterator>;

        // 24.3.11.2, construct/copy/destroy
        constexpr non_invalidating_vector() noexcept(noexcept(Allocator())) : non_
        constexpr explicit non_invalidating_vector(const Allocator&) noexcept;
        constexpr explicit non_invalidating_vector(size_type n, const Allocator& =
        constexpr non_invalidating_vector(size_type n, const T& value, const Alloc
        template<class InputIterator>
        constexpr non_invalidating_vector(InputIterator first, InputIterator last,
        template<container-compatible-range <T> R>
        constexpr non_invalidating_vector(from_range_t, R&& rg, const Allocator& =
        constexpr non_invalidating_vector(const non_invalidating_vector& x);
        constexpr non_invalidating_vector(non_invalidating_vector&&) noexcept;
        constexpr non_invalidating_vector(const non_invalidating_vector&, const ty
        constexpr non_invalidating_vector(non_invalidating_vector&&, const type_ic
        constexpr non_invalidating_vector(initializer_list<T>, const Allocator& =
        constexpr ~non_invalidating_vector();
        constexpr allocator_type get_allocator() const noexcept;

        // iterators
        constexpr iterator begin() noexcept;
        constexpr const_iterator begin() const noexcept;
        constexpr iterator end() noexcept;
        constexpr const_iterator end() const noexcept;
        constexpr reverse_iterator rbegin() noexcept;
        constexpr const_reverse_iterator rbegin() const noexcept;
        constexpr reverse_iterator rend() noexcept;
        constexpr const_reverse_iterator rend() const noexcept;

        constexpr const_iterator cbegin() const noexcept;
        constexpr const_iterator cend() const noexcept;
        constexpr const_reverse_iterator crbegin() const noexcept;

```

```

constexpr const_reverse_iterator crend() const noexcept;

// 24.3.11.3, capacity
[[nodiscard]] constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;
constexpr size_type capacity() const noexcept;

// element access
constexpr reference operator[](size_type n);
constexpr const_reference operator[](size_type n) const;
constexpr const_reference at(size_type n) const;
constexpr reference at(size_type n);
constexpr reference front();
constexpr const_reference front() const;
constexpr reference back();
constexpr const_reference back() const;

// 24.3.11.4, data access
constexpr T* data() noexcept;
constexpr const T* data() const noexcept;
};

template<class InputIterator, class Allocator = allocator<iter-value-type <InputIterator>,
        non_invalidating_vector(InputIterator, InputIterator, Allocator = Allocator())
    -> non_invalidating_vector<iter-value-type <InputIterator>, Allocator>;

template<ranges::input_range R, class Allocator = allocator<ranges::range_value_t<R>,
        non_invalidating_vector(from_range_t, R&&, Allocator = Allocator())
    -> non_invalidating_vector<ranges::range_value_t<R>, Allocator>;
}

```

⁴ An incomplete type T may be used when instantiating non_invalidating_vector if the allocator meets the allocator completeness requirements (16.4.4.6.2). T shall be complete before any member of the resulting specialization of non_invalidating_vector is referenced.

24.3.11.2 Constructors [non_invalidating_vector.cons]

```
constexpr explicit non_invalidating_vector(const Allocator&) noexcept;
```

¹ Effects: Constructs an empty non_invalidating_vector, using the specified allocator.

² Complexity: Constant.

```
constexpr explicit non_invalidating_vector(size_type n, const Allocator& = Allocat
```

³ Preconditions: T is Cpp17DefaultInsertable into *this.

⁴ Effects: Constructs a non_invalidating_vector with n default-inserted elements using the specified allocator.

⁵ Complexity: Linear in n.

```
constexpr non_invalidating_vector(size_type n, const T& value,  
    const Allocator& = Allocator());
```

⁶ Preconditions: T is Cpp17CopyInsertable into *this.

⁷ Effects: Constructs a non_invalidating_vector with n copies of value, using the specified allocator.

⁸ Complexity: Linear in n.

```
template<class InputIterator>  
constexpr non_invalidating_vector(InputIterator first, InputIterator last,  
    const Allocator& = Allocator());
```

⁹ Effects: Constructs a non_invalidating_vector equal to the range [first, last), using the specified allocator.

¹⁰ Complexity: Makes only N calls to the copy constructor of T (where N is the distance between first and last) and no reallocations if iterators first and last are of forward, bidirectional, or random access categories. It makes order N calls to the copy constructor of T and order log N reallocations if they are just input iterators.

```
template<container-compatible-range <T> R>  
constexpr non_invalidating_vector(from_range_t, R&& rg, const Allocator& = Allocat
```

¹¹ Effects: Constructs a non_invalidating_vector object with the elements of the range rg, using the specified allocator.

¹² *Complexity*: Initializes exactly N elements from the results of dereferencing successive iterators of `rg`, where N is `ranges::distance(rg)`. Performs no reallocations if R models `ranges::forward_range` or `ranges::sized_range`; otherwise, performs order $\log N$ reallocations and order N calls to the copy or move constructor of T.

24.3.11.3 Capacity [`non_invalidating_vector.capacity`]

```
constexpr size_type capacity() const noexcept;
```

¹ *Returns*: The total number of elements that the `non_invalidating_vector` can hold without requiring reallocation.

² *Complexity*: Constant time.

24.3.11.4 Data [`non_invalidating_vector.data`]

```
constexpr T* data() noexcept;  
constexpr const T* data() const noexcept;
```

¹ *Returns*: A pointer such that `[data(), data() + size())` is a valid range. For a non-empty `non_invalidating_vector`, `data() == addressof(front())` is true.

² *Complexity*: Constant time.

24.3.11 Class template vector [`vector`]

24.3.11.1 Overview [`vector.overview`]

¹ A `vector` is a sequence container that supports (amortized) constant time `insert` and `erase` operations at the end; `insert` and `erase` in the middle take linear time. Storage management is handled automatically, though hints can be given to improve efficiency.

² A `vector` meets all of the requirements of a container (24.2.2.2), of a reversible container (24.2.2.3), of an allocator-aware container (24.2.2.5), of a sequence container, including most of the optional sequence container requirements (24.2.4), and, for an element type other than `bool`, of a contiguous container (24.2.2.2). The exceptions are the `push_front`, `prepend_range`, `pop_front`, and `emplace_front` member functions, which are not provided. Descriptions are provided here only for operations on `vector` that are not described in one of these tables or for operations where there is additional semantic information.

³ The types `iterator` and `const_iterator` meet the `constexpr` iterator requirements (25.3.1).


```

namespace std {
    template<class T, class Allocator = allocator<T>>
    class vector : public non_invalidating_vector {
    public:
        // 24.3.11.2, construct/copy/destroy
        constexpr vector() noexcept(noexcept(Allocator())) : vector(Allocator()) {}
        constexpr vector(const vector& x);
        constexpr vector(vector&&) noexcept;
        constexpr vector(const vector&, const type_identity_t<Allocator>&);
        constexpr vector(vector&&, const type_identity_t<Allocator>&);
        using non_invalidating_vector::non_invalidating_vector;
        constexpr ~vector();
        constexpr vector& operator=(const vector& x);
        constexpr vector& operator=(vector&& x)
            noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment()
                || allocator_traits<Allocator>::is_always_equal::value);
        constexpr vector& operator=(initializer_list<T>);
        template<class InputIterator>
        constexpr void assign(InputIterator first, InputIterator last);
        template<container-compatible-range <T> R>
        constexpr void assign_range(R&& rg);
        constexpr void assign(size_type n, const T& u);
        constexpr void assign(initializer_list<T>);

        // 24.3.11.3, capacity
        constexpr void resize(size_type sz);
        constexpr void resize(size_type sz, const T& c);
        constexpr void reserve(size_type n);
        constexpr void shrink_to_fit();

        // 24.3.11.5, modifiers
        template<class... Args> constexpr reference emplace_back(Args&&... args);
        constexpr void push_back(const T& x);
        constexpr void push_back(T&& x);
        template<container-compatible-range <T> R>
        constexpr void append_range(R&& rg);
        constexpr void pop_back();

        template<class... Args> constexpr iterator emplace(const_iterator position,
            Args&&... args);
        constexpr iterator insert(const_iterator position, const T& x);
        constexpr iterator insert(const_iterator position, T&& x);
        constexpr iterator insert(const_iterator position, size_type n, const T& x);
        template<class InputIterator>
        constexpr iterator insert(const_iterator position,
            InputIterator first, InputIterator last);
        template<container-compatible-range <T> R>
        constexpr iterator insert_range(const_iterator position, R&& rg);
        constexpr iterator insert(const_iterator position, initializer_list<T> il);
        constexpr iterator erase(const_iterator position);

```

```

        constexpr iterator erase(const_iterator first, const_iterator last);
        constexpr void swap(vector&)
            noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value
                allocator_traits<Allocator>::is_always_equal::value);

        constexpr void clear() noexcept;
};

template<class InputIterator, class Allocator = allocator<iter-value-type <InputIterator>,
    vector(InputIterator, InputIterator, Allocator = Allocator())
    -> vector<iter-value-type <InputIterator>, Allocator>;

template<ranges::input_range R, class Allocator = allocator<ranges::range_value_t<R>,
    vector(from_range_t, R&&, Allocator = Allocator())
    -> vector<ranges::range_value_t<R>, Allocator>;
}

```

⁴ An incomplete type T may be used when instantiating `vector` if the allocator meets the allocator completeness requirements (16.4.4.6.2). T shall be complete before any member of the resulting specialization of `vector` is referenced.

24.3.11.2 Constructors [vector.cons]

```
constexpr explicit vector(const Allocator&) noexcept;
```

¹~~Effects:~~ Constructs an empty vector, using the specified allocator.

²~~Complexity:~~ Constant.

```
constexpr explicit vector(size_type n, const Allocator& = Allocator());
```

³~~Preconditions:~~ T is Cpp17DefaultInsertable into *this.

⁴~~Effects:~~ Constructs a vector with n default-inserted elements using the specified allocator.

⁵~~Complexity:~~ Linear in n.

```
constexpr vector(size_type n, const T& value,
    const Allocator& = Allocator());
```

⁶~~Preconditions:~~ T is Cpp17CopyInsertable into *this.

⁷~~Effects: Constructs a vector with n copies of value, using the specified allocator.~~

⁸~~Complexity: Linear in n.~~

```
template<class InputIterator>
constexpr vector(InputIterator first, InputIterator last,
    const Allocator& = Allocator());
```

⁹~~Effects: Constructs a vector equal to the range [first, last), using the specified allocator.~~

¹⁰~~Complexity: Makes only N calls to the copy constructor of T (where N is the distance between first and last) and no reallocations if iterators first and last are of forward, bidirectional, or random access categories. It makes order N calls to the copy constructor of T and order log N reallocations if they are just input iterators.~~

```
template<container-compatible-range <T> R>
constexpr vector(from_range_t, R&& rg, const Allocator& = Allocator());
```

¹¹ *Effects:* Constructs a `vector` object with the elements of the range `rg`, using the specified allocator.

¹² *Complexity:* Initializes exactly N elements from the results of dereferencing successive iterators of `rg`, where N is `ranges::distance(rg)`. Performs no reallocations if R models `ranges::forward_range` or `ranges::sized_range`; otherwise, performs order log N reallocations and order N calls to the copy or move constructor of T.

vector inherits the constructors of non invalidating vector.

24.3.11.3 Capacity [`vector.capacity`]

```
constexpr size_type capacity() const noexcept;
```

¹~~Returns: The total number of elements that the vector can hold without requiring reallocation.~~

²~~Complexity: Constant time.~~

```
constexpr void reserve(size_type n);
```

³¹ *Preconditions:* T is `Cpp17MoveInsertable` into *this.

⁴² *Effects*: A directive that informs a vector of a planned change in size, so that it can manage the storage allocation accordingly. After `reserve()`, `capacity()` is greater or equal to the argument of `reserve` if reallocation happens; and equal to the previous value of `capacity()` otherwise. Reallocation happens at this point if and only if the current capacity is less than the argument of `reserve()`. If an exception is thrown other than by the move constructor of a non-`Cpp17CopyInsertable` type, there are no effects.

⁵³ *Throws*: `length_error` if `n > max_size()`.²⁰⁷

⁶⁴ *Complexity*: It does not change the size of the sequence and takes at most linear time in the size of the sequence.

⁷⁵ *Remarks*: Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence, as well as the past-the-end iterator.

[*Note 1*: If no reallocation happens, they remain valid. — *end note*]

No reallocation shall take place during insertions that happen after a call to `reserve()` until an insertion would make the size of the `vector` greater than the value of `capacity()`.

```
constexpr void shrink_to_fit();
```

⁸⁶ *Preconditions*: `T` is `Cpp17MoveInsertable` into `*this`.

⁹⁷ *Effects*: `shrink_to_fit` is a non-binding request to reduce `capacity()` to `size()`.

[*Note 2*: The request is non-binding to allow latitude for implementation-specific optimizations. — *end note*]

It does not increase `capacity()`, but may reduce `capacity()` by causing reallocation. If an exception is thrown other than by the move constructor of a non-`Cpp17CopyInsertable` `T` there are no effects.

¹⁰⁸ *Complexity*: If reallocation happens, linear in the size of the sequence.

¹⁴⁹ *Remarks*: Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence as well as the past-the-end iterator.

[*Note 3*: If no reallocation happens, they remain valid. — *end note*]

```
constexpr void swap(vector& x)
noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
         allocator_traits<Allocator>::is_always_equal::value);
```

¹²¹⁰ *Effects*: Exchanges the contents and `capacity()` of `*this` with that of `x`.

¹³¹¹ *Complexity*: Constant time.

```
constexpr void resize(size_type sz);
```

¹⁴¹² *Preconditions*: `T` is `Cpp17MoveInsertable` and `Cpp17DefaultInsertable` into `*this`.

¹⁵¹³ *Effects*: If `sz < size()`, erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` default-inserted elements to the sequence.

¹⁶¹⁴ *Remarks*: If an exception is thrown other than by the move constructor of a non-`Cpp17CopyInsertable` `T` there are no effects.

```
constexpr void resize(size_type sz, const T& c);
```

¹⁷¹⁵ *Preconditions*: `T` is `Cpp17CopyInsertable` into `*this`.

¹⁸¹⁶ *Effects*: If `sz < size()`, erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` copies of `c` to the sequence.

¹⁹¹⁷ *Remarks*: If an exception is thrown there are no effects.

24.3.11.4 Data [vector.data]

```
constexpr T* data() noexcept;  
constexpr const T* data() const noexcept;
```

¹ ~~*Returns*: A pointer such that `[data(), data() + size())` is a valid range. For a non-empty vector, `data() == addressof(front())` is true.~~

² ~~*Complexity*: Constant time.~~

~~207) `reserve()` uses `Allocator::allocate()` which can throw an appropriate exception.~~

24.3.11.54 Modifiers [vector.modifiers]

```

constexpr iterator insert(const_iterator position, const T& x);
constexpr iterator insert(const_iterator position, T&& x);
constexpr iterator insert(const_iterator position, size_type n, const T& x);
template<class InputIterator>
constexpr iterator insert(const_iterator position, InputIterator first, InputIterator last);
template<container-compatible-range <T> R>
constexpr iterator insert_range(const_iterator position, R&& rg);
constexpr iterator insert(const_iterator position, initializer_list<T>);
template<class... Args> constexpr reference emplace_back(Args&&... args);
template<class... Args> constexpr iterator emplace(const_iterator position, Args&&... args);
constexpr void push_back(const T& x);
constexpr void push_back(T&& x);
template<container-compatible-range <T> R>
constexpr void append_range(R&& rg);

```

¹ *Complexity*: If reallocation happens, linear in the number of elements of the resulting vector; otherwise, linear in the number of elements inserted plus the distance to the end of the vector .

² *Remarks*: Causes reallocation if the new size is greater than the old capacity. Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence, as well as the past-the-end iterator. If no reallocation happens, then references, pointers, and iterators before the insertion point remain valid but those at or after the insertion point, including the past-the-end iterator, are invalidated. If an exception is thrown other than by the copy constructor, move constructor, assignment operator, or move assignment operator of T or by any InputIterator operation there are no effects. If an exception is thrown while inserting a single element at the end and T is

Cpp17CopyInsertable or is_nothrow_move_constructible_v<T> is true, there are no effects. Otherwise, if an exception is thrown by the move constructor of a non- Cpp17CopyInsertable T, the effects are unspecified.

```

constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void pop_back();

```

³ *Effects*: Invalidates iterators and references at or after the point of the erase.

⁴ *Throws*: Nothing unless an exception is thrown by the assignment operator or move assignment operator of T.

⁵ *Complexity*: The destructor of T is called the number of times equal to the number of the elements erased, but the assignment operator of T is called the number of times equal to the number of elements in the vector after the erased elements.

24.3.11.65 Erasure [vector.erase]

```
template<class T, class Allocator, class U = T>
constexpr typename vector<T, Allocator>::size_type
erase(vector<T, Allocator>& c, const U& value);
```

¹ *Effects*: Equivalent to:

```
auto it = remove(c.begin(), c.end(), value);
auto r = distance(it, c.end());
c.erase(it, c.end());
return r;
```

```
template<class T, class Allocator, class Predicate>
constexpr typename vector<T, Allocator>::size_type
erase_if(vector<T, Allocator>& c, Predicate pred);
```

² *Effects*: Equivalent to:

```
auto it = remove_if(c.begin(), c.end(), pred);
auto r = distance(it, c.end());
c.erase(it, c.end());
return r;
```

Feature-test macro [version.syn]

Add the following macro definition to [version.syn], header <version> synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_non_invalidating_vector 20XXXXL // also in <ranges>, <tuple>, <u
```

Impact on the standard

The non alternative is a pure library extension, affecting no other parts of the library or language.

The better inheritance based alternative is a refactor of the `std::vector` moving some of its methods into a new base class. It does not effect other parts of the library or language. As `std::vector` won't be losing any functionality than this is not expected to be a breaking change for users of `std::vector` .

The proposed changes are relative to the current working draft N4981 ^[4].

Implementation

```
template<
    class T,
    class Allocator = std::allocator<T>
>
class non_invalidating_vector_ref
{
public:
    // Member types
    typedef std::vector<T, Allocator> actual_type;

    typedef actual_type::value_type value_type;
    typedef actual_type::allocator_type allocator_type;
    typedef actual_type::size_type size_type;
    typedef actual_type::difference_type difference_type;
    typedef actual_type::reference reference;
    typedef actual_type::const_reference const_reference;
    typedef actual_type::pointer pointer;
    typedef actual_type::const_pointer const_pointer;
    typedef actual_type::iterator iterator;
    typedef actual_type::const_iterator const_iterator;
    typedef actual_type::reverse_iterator reverse_iterator;
    typedef actual_type::const_reverse_iterator const_reverse_iterator;
private:
    actual_type& ref;
public:
    constexpr non_invalidating_vector_ref() = delete;
    constexpr non_invalidating_vector_ref(const non_invalidating_vector_ref&) = delete;
    constexpr non_invalidating_vector_ref(non_invalidating_vector_ref&&) = default;
    constexpr non_invalidating_vector_ref operator=(const non_invalidating_vector_ref&) = delete;
    constexpr non_invalidating_vector_ref operator=(const non_invalidating_vector_ref&&) = delete;

    constexpr non_invalidating_vector_ref(actual_type& reference) : ref{reference} {}

    //constexpr non_invalidating_vector_ref& operator=( const non_invalidating_vector_ref& other ) = delete;
    constexpr non_invalidating_vector_ref& operator=( non_invalidating_vector_ref&& other ) = delete;
    constexpr non_invalidating_vector_ref& operator=( std::initializer_list<value_type> ilist ) = delete;
    constexpr void assign( size_type count, const T& value ) = delete;
    template< class InputIt >
    constexpr void assign( InputIt first, InputIt last ) = delete;
    constexpr void assign( std::initializer_list<T> ilist ) = delete;
    template< class R >
    constexpr void assign_range( R&& rg ) = delete;
    constexpr allocator_type get_allocator() const
    {
        return ref.get_allocator();
    }
}
```

```

// Element access
constexpr reference at( size_type pos )
{
    return ref.at(pos);
}
constexpr const_reference at( size_type pos ) const
{
    return ref.at(pos);
}
constexpr reference operator[]( size_type pos )
{
    return ref[pos];
}
constexpr const_reference operator[]( size_type pos ) const
{
    return ref[pos];
}
constexpr reference front()
{
    return ref.front();
}
constexpr const_reference front() const
{
    return ref.front();
}
constexpr reference back()
{
    return ref.back();
}
constexpr const_reference back() const
{
    return ref.back();
}
constexpr T* data()
{
    return ref.data();
}
constexpr const T* data() const
{
    return ref.data();
}
// Iterators
constexpr iterator begin()
{
    return ref.begin();
}
constexpr const_iterator begin() const
{
    return ref.begin();
}

```

```

}
constexpr const_iterator cbegin() const noexcept
{
    return ref.cbegin();
}
constexpr iterator end() noexcept
{
    return ref.end();
}
constexpr const_iterator end() const noexcept
{
    return ref.end();
}
constexpr const_iterator cend() const noexcept
{
    return ref.cend();
}
constexpr reverse_iterator rbegin()
{
    return ref.rbegin();
}
constexpr const_reverse_iterator rbegin() const
{
    return ref.rbegin();
}
constexpr const_reverse_iterator crbegin() const noexcept
{
    return ref.crbegin();
}
constexpr reverse_iterator rend()
{
    return ref.rend();
}
constexpr const_reverse_iterator rend() const
{
    return ref.rend();
}
constexpr const_reverse_iterator crend() const noexcept
{
    return ref.crend();
}
// Capacity
constexpr bool empty() const
{
    return ref.empty();
}
constexpr size_type size() const
{
    return ref.size();
}

```

```

    }
    constexpr size_type max_size() const
    {
        return ref.max_size();
    }
    constexpr size_type capacity() const
    {
        return ref.capacity();
    }
    void shrink_to_fit() = delete;
    // Modifiers
    constexpr void clear() = delete;
    constexpr iterator insert( const_iterator pos, const T& value ) = delete;
    constexpr iterator insert( const_iterator pos, T&& value ) = delete;
    constexpr iterator insert( const_iterator pos, size_type count, const T& value
template< class InputIt >
    constexpr iterator insert( const_iterator pos, InputIt first, InputIt last ) =
    constexpr iterator insert( const_iterator pos, std::initializer_list<T> ilist
template< class R >
    constexpr iterator insert_range( const_iterator pos, R&& rg ) = delete;
template< class... Args >
    constexpr iterator emplace( const_iterator pos, Args&&... args ) = delete;
    constexpr iterator erase( const_iterator pos ) = delete;
    constexpr iterator erase( const_iterator first, const_iterator last ) = delete;
    constexpr void push_back( const T& value ) = delete;
    constexpr void push_back( T&& value ) = delete;
template< class... Args >
    constexpr reference emplace_back( Args&&... args ) = delete;
template< class R >
    constexpr void append_range( R&& rg ) = delete;
    constexpr void pop_back() = delete;
    constexpr void resize( size_type count ) = delete;
    constexpr void resize( size_type count, const value_type& value ) = delete;
    constexpr void swap( actual_type& other ) noexcept(std::allocator_traits<Alloc

operator const actual_type&() const
{
    return ref;
}
};

template< class T, class Alloc >
constexpr bool operator==( const non_invalidating_vector_ref<T, Alloc>& lhs,
                           const non_invalidating_vector_ref<T, Alloc>& rhs )
{
    return lhs.ref == rhs.ref;
}

template< class T, class Alloc >

```

```

constexpr auto/*std::synth-three-way-result<T>*/
    operator<=>( const non_invalidating_vector_ref<T, Alloc>& lhs,
                 const non_invalidating_vector_ref<T, Alloc>& rhs )
{
    return lhs.ref <=> rhs.ref;
}

template< class T, class Alloc >
void swap( non_invalidating_vector_ref<T, Alloc>& lhs, non_invalidating_vector_ref<T, Alloc>& rhs )
{
    lhs.ref.swap( rhs.ref );
}

template< class T, class Alloc, class U = T >
constexpr non_invalidating_vector_ref<T, Alloc>::size_type erase( non_invalidating_vector_ref<T, Alloc>& v,
                                                                    const_iterator pos )
{
    return v.ref.erase( pos );
}

template< class T, class Alloc, class Pred >
constexpr non_invalidating_vector_ref<T, Alloc>::size_type erase_if( non_invalidating_vector_ref<T, Alloc>& v,
                                                                      const_iterator pos, const_iterator end )
{
    return v.ref.erase_if( pos, end, Pred() );
}

template<
    class T,
    class Allocator = std::allocator<T>,
    class Container = std::vector<T, Allocator>
>
class non_invalidating_vector
{
public:
    // Member types
    typedef std::vector<T, Allocator> actual_type;

    typedef actual_type::value_type value_type;
    typedef actual_type::allocator_type allocator_type;
    typedef actual_type::size_type size_type;
    typedef actual_type::difference_type difference_type;
    typedef actual_type::reference reference;
    typedef actual_type::const_reference const_reference;
    typedef actual_type::pointer pointer;
    typedef actual_type::const_pointer const_pointer;
    typedef actual_type::iterator iterator;
    typedef actual_type::const_iterator const_iterator;
    typedef actual_type::reverse_iterator reverse_iterator;
    typedef actual_type::const_reverse_iterator const_reverse_iterator;

private:
    actual_type inner;

public:
    constexpr non_invalidating_vector() noexcept(noexcept(Allocator()))
        : inner() {}
    constexpr explicit non_invalidating_vector( const Allocator& alloc ) noexcept
        : inner(alloc) {}
    constexpr non_invalidating_vector( size_type count, const T& value, const Alloc& alloc )
        : inner(count, value, alloc) {}
    explicit non_invalidating_vector( size_type count, const Allocator& alloc = Alloc() )
        : inner(count, T(), alloc) {}

```

```

        : inner(count, alloc) {}
template< class InputIt >
constexpr non_invalidating_vector( InputIt first, InputIt last, const Allocator& alloc )
    : inner(first, last, alloc) {}
constexpr non_invalidating_vector( const non_invalidating_vector& other )
    : inner(other) {}
constexpr non_invalidating_vector( const non_invalidating_vector& other, const Allocator& alloc )
    : inner(other, alloc) {}
constexpr non_invalidating_vector( non_invalidating_vector&& other ) noexcept
    : inner(other) {}
constexpr non_invalidating_vector( non_invalidating_vector&& other, const Allocator& alloc )
    : inner(other, alloc) {}
constexpr non_invalidating_vector( std::initializer_list<T> init, const Allocator& alloc )
    : inner(init, alloc) {}
template< class R >
constexpr non_invalidating_vector( std::from_range_t, R&& rg, const Allocator& alloc )
    : inner(std::from_range, rg, alloc) {}

constexpr allocator_type get_allocator() const
{
    return inner.get_allocator();
}
// Element access
constexpr reference at( size_type pos )
{
    return inner.at(pos);
}
constexpr const_reference at( size_type pos ) const
{
    return inner.at(pos);
}
constexpr reference operator[]( size_type pos )
{
    return inner[pos];
}
constexpr const_reference operator[]( size_type pos ) const
{
    return inner[pos];
}
constexpr reference front()
{
    return inner.front();
}
constexpr const_reference front() const
{
    return inner.front();
}
constexpr reference back()
{

```

```

        return inner.back();
    }
constexpr const_reference back() const
{
    return inner.back();
}
constexpr T* data()
{
    return inner.data();
}
constexpr const T* data() const
{
    return inner.data();
}
// Iterators
constexpr iterator begin()
{
    return inner.begin();
}
constexpr const_iterator begin() const
{
    return inner.begin();
}
constexpr const_iterator cbegin() const noexcept
{
    return inner.cbegin();
}
constexpr iterator end() noexcept
{
    return inner.end();
}
constexpr const_iterator end() const noexcept
{
    return inner.end();
}
constexpr const_iterator cend() const noexcept
{
    return inner.cend();
}
constexpr reverse_iterator rbegin()
{
    return inner.rbegin();
}
constexpr const_reverse_iterator rbegin() const
{
    return inner.rbegin();
}
constexpr const_reverse_iterator crbegin() const noexcept
{

```

```

        return inner.crbegin();
    }
    constexpr reverse_iterator rend()
    {
        return inner.rend();
    }
    constexpr const_reverse_iterator rend() const
    {
        return inner.rend();
    }
    constexpr const_reverse_iterator crend() const noexcept
    {
        return inner.crend();
    }
    // Capacity
    constexpr bool empty() const
    {
        return inner.empty();
    }
    constexpr size_type size() const
    {
        return inner.size();
    }
    constexpr size_type max_size() const
    {
        return inner.max_size();
    }
    constexpr size_type capacity() const
    {
        return inner.capacity();
    }
    void shrink_to_fit() = delete;
    // Modifiers
    constexpr void clear() = delete;
    constexpr iterator insert( const_iterator pos, const T& value ) = delete;
    constexpr iterator insert( const_iterator pos, T&& value ) = delete;
    constexpr iterator insert( const_iterator pos, size_type count, const T& value ) = delete;
    template< class InputIt >
    constexpr iterator insert( const_iterator pos, InputIt first, InputIt last ) = delete;
    constexpr iterator insert( const_iterator pos, std::initializer_list<T> ilist ) = delete;
    template< class R >
    constexpr iterator insert_range( const_iterator pos, R&& rg ) = delete;
    template< class... Args >
    constexpr iterator emplace( const_iterator pos, Args&&... args ) = delete;
    constexpr iterator erase( const_iterator pos ) = delete;
    constexpr iterator erase( const_iterator first, const_iterator last ) = delete;
    constexpr void push_back( const T& value ) = delete;
    constexpr void push_back( T&& value ) = delete;
    template< class... Args >

```



```

constexpr reference emplace_back( Args&&... args ) = delete;
template< class R >
constexpr void append_range( R&& rg ) = delete;
constexpr void pop_back() = delete;
constexpr void resize( size_type count ) = delete;
constexpr void resize( size_type count, const value_type& value ) = delete;
constexpr void swap( actual_type& other ) noexcept(std::allocator_traits<Alloc

operator non_invalidating_vector_ref<T, Allocator>()
{
    return non_invalidating_vector_ref<T, Allocator>{inner};
}

operator const actual_type&() const
{
    return inner;
}
};

template< class T, class Alloc >
constexpr bool operator==( const non_invalidating_vector<T, Alloc>& lhs,
                           const non_invalidating_vector<T, Alloc>& rhs )
{
    return lhs.ref == rhs.ref;
}

template< class T, class Alloc >
constexpr auto/*std::synth-three-way-result<T>*/
operator<=>( const non_invalidating_vector<T, Alloc>& lhs,
            const non_invalidating_vector<T, Alloc>& rhs )
{
    return lhs.ref <=> rhs.ref;
}

template< class T, class Alloc >
void swap( non_invalidating_vector<T, Alloc>& lhs, non_invalidating_vector<T, Alloc

template< class T, class Alloc, class U = T >
constexpr non_invalidating_vector<T, Alloc>::size_type erase( non_invalidating_vec

template< class T, class Alloc, class Pred >
constexpr non_invalidating_vector<T, Alloc>::size_type erase_if( non_invalidating_

```

References

1. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3274r0.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3274r0.pdf>)  
2. <https://github.com/microsoft/GSL> (<https://github.com/microsoft/GSL>) 
3. <https://github.com/gsl-lite/gsl-lite> (<https://github.com/gsl-lite/gsl-lite>) 
4. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/n4981.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/n4981.pdf>) 